

BEATLE 実機テスト進行マニュアル

事前準備: 絶対に確認すること

- バッテリーの確認: モータの速度は電池残量に直結します。必ず「満充電」の電池でテストを開始してください。
- オフセットの確認: BEALEがまっすぐ走れるように設定を行う。
- コースの用意: マップ通り、オーバル状のメインラインと十字の交差点を用意します。

【Step 1】基礎ラインレースの調整(P制御のチューニング)

目的: ロボットがオーバルコースを脱線せずに、滑らかに1周できるようにする。

1. プログラムの準備:
 - 現在の `parking.ino` のまま(昨日追加した交差点機能などはすべてコメントアウトした状態)機体には書き込みます。
2. 定数の確認:
 - 直進のベース速度が `baseSpeedL = 150, baseSpeedR = 30` になっていることを確認します。
 - 積分ゲインは必ず `Ki = 0.00` にしておきます。
3. **Kp**(比例ゲイン)の調整:
 - ロボットをライン上に置き、走らせます。
 - カーブで曲がりきれず脱線する場合: **Kp** の値を少し大きくします(例: 0.05 → 0.07)。
 - 直線で左右に激しく首を振る(ガタガタ震える)場合: **Kp** の値を少し小さくします(例: 0.05 → 0.04)。
4. クリア条件: コースをスムーズに何周も走り続けられるようになったらStep 1クリアです。

【Step 2】交差点検知の「閾値(しきいち)」を測る

目的: ロボットが「ここは十字マークだ!」と認識するための基準値(`CROSS_THRESHOLD`)を決定する。

1. センサ値の計測:
 - プログラムを書き換える前に、ロボットを十字マーク(交差点)の真上に手で置きます。
 - サンプルコードの `LT_sensor_test.ino` のように、シリアルモニタを開いて6つのセンサが真っ黒の線の上にいるときの値を確認します。
2. 閾値の計算:
 - 6つのセンサ値の合計を計算します(例: すべて黒なら $500 \times 6 = 3000$ など)。
 - その合計値より「少しだけ小さい値(例: 2800)」を、プログラムの `CROSS_THRESHOLD` に設定します。
3. クリア条件: 確実に交差点とそうでない場所の数値の差が把握できたらStep 2クリアです。

【Step 3】交差点での「停止」テスト

目的: ライントレース中に十字マークを検知して、自律的にピタッと止まるか確認する。

1. プログラムの解放(コメントアウト解除):

- `parking.ino` 内の【追加・解除箇所A】(`CROSS_THRESHOLD`の定義)、【追加・解除箇所B】(`isCrossroad`関数)、【追加・解除箇所C】(`loop`内のif文) のコメントアウト(`/*` と `*/`)を外して有効化し、機体に書き込みます。
2. テスト走行:
 - ロボットをコースに置き、走らせます。
 - 素通りしてしまう場合: `CROSS_THRESHOLD` の値が高すぎるので、少し下げます。
 - 何もない直線で急停止してしまう場合: `CROSS_THRESHOLD` の値が低すぎるので、少し上げます。
 3. クリア条件: 何度やっても、十字マークを踏んだ瞬間にモータが停止 (90, 90)して動けなくなればStep 3クリアです。

【Step 4】アライメント(姿勢補正)のテスト

目的: 旋回後に姿勢が斜めにズれていても、自力で真っ直ぐに直せるか確認する。

1. プログラムの解放:
 - 【追加・解除箇所D】(`alignToLine`関数) と【追加・解除箇所E】(`turn90Right`関数) のコメントアウトを外します。
 - `loop()` の停止処理の直後に、テストとしてこの2つの関数を呼び出すように書き換えます。
(例: 停止 → `delay(1000)` → `turn90Right()` → `alignToLine()` → 再び無限ループで停止)
2. テスト走行:
 - 十字マークで止まった後、右に旋回し、その直後にロボットが「ピクピク」と動いて、十字の線に対してビシッと垂直な姿勢になれば成功です。
3. アライメント用ゲインの調整:
 - 姿勢が戻りきらない、または揺れ続ける場合は、アライメント専用の `Kp_align` の数値を調整します。
4. クリア条件: 斜めに置かれた状態からでも、一瞬で真っ直ぐな姿勢を復元できればStep 4クリアです。